```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

print("OpenCV Version:", cv2.__version__)
print("Libraries imported successfully!")
```

```
OpenCV Version: 5.0.0
Libraries imported successfully!
```

```
uploaded = files.upload()

video_path = list(uploaded.keys())[0]

print("Uploaded video:", video_path)
```

Choose Files 278ee15f-a…b15e49.mp4

278ee15f-a055-469b-a256-798d36b15e49.mp4(video/mp4) - 1793714 bytes, last modified: 9/17/2026 - 100% done

```
Saving 278ee15f-a055-469b-a256-798d36b15e49.mp4 to 278ee15f-a055-469b-a256-798d36b15e49
Uploaded video: 278ee15f-a055-469b-a256-798d36b15e49.mp4
```

```
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open video.")
else:
    print("Video opened successfully!")

fps = cap.get(cv2.CAP_PROP_FPS)
frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

print("FPS:", fps)
print("Total Frames:", frame_count)
print("Width:", width)
print("Height:", height)
```

```
Video opened successfully!
FPS: 24.0
Total Frames: 73
Width: 960
Height: 960
```

```
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

ret, frame1 = cap.read()
ret2, frame2 = cap.read()

if not ret or not ret2:
    print("Could not read frames.")
else:
    print("Two consecutive frames loaded successfully.")
```

```
frame1_rgb = cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB)
frame2_rgb = cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB)

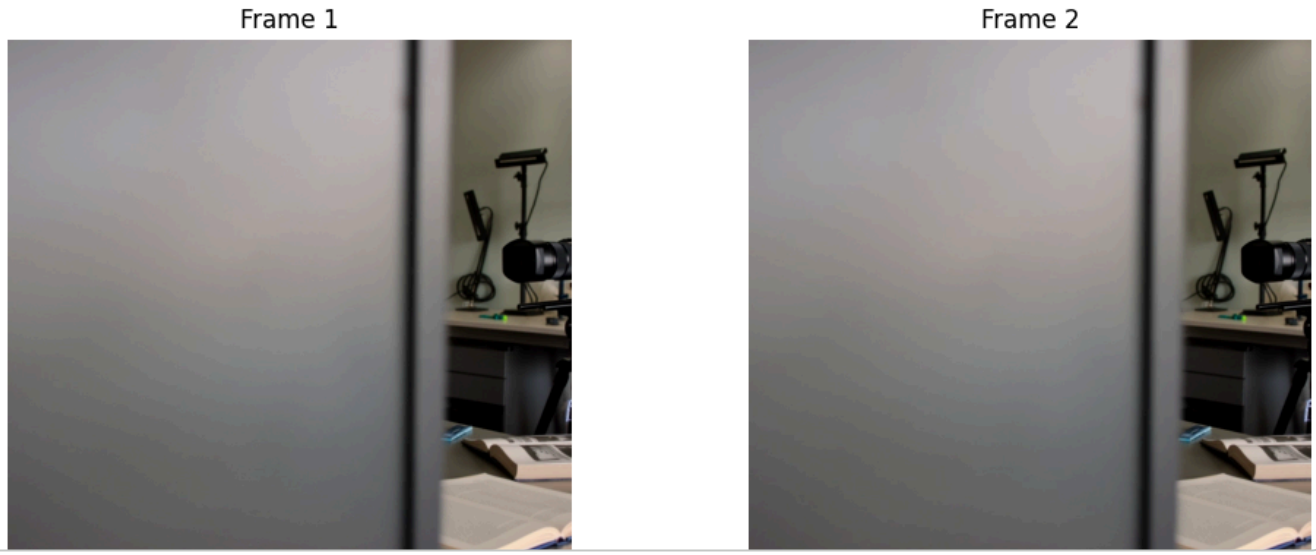
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(frame1_rgb)
plt.title("Frame 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(frame2_rgb)
plt.title("Frame 2")
plt.axis("off")

plt.show()
```

```
Two consecutive frames loaded successfully.
```



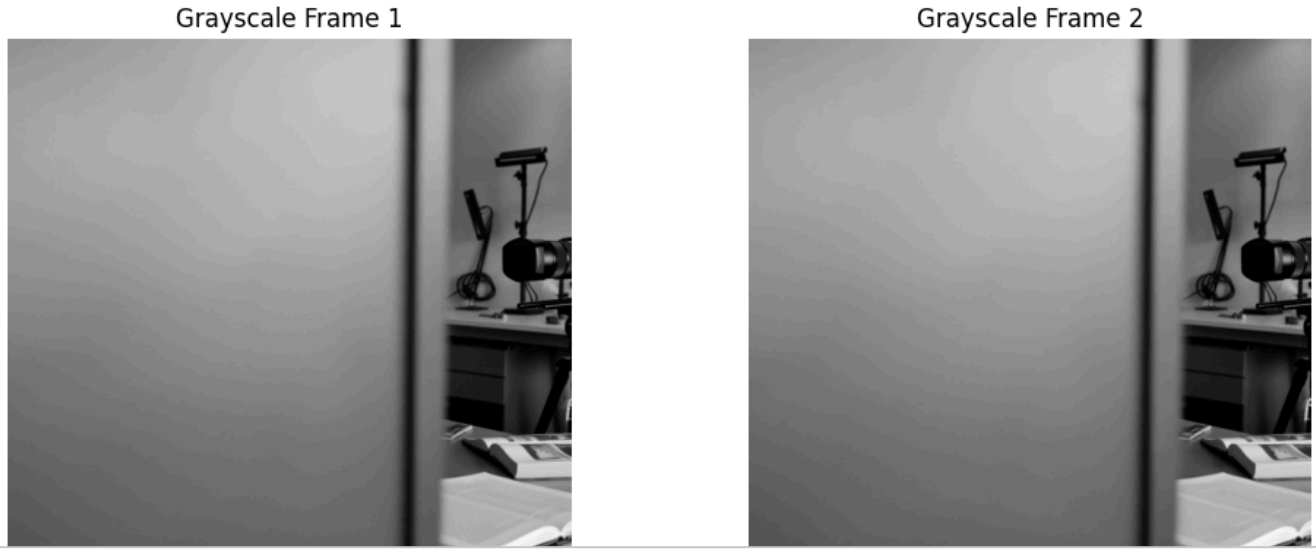

```
gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray1, cmap='gray')
plt.title("Grayscale Frame 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gray2, cmap='gray')
plt.title("Grayscale Frame 2")
plt.axis("off")

plt.show()
```




```
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

p0 = cv2.goodFeaturesToTrack(
    gray1,
    mask=None,
    **feature_params
)

if p0 is not None:
    print("Number of feature points detected:", len(p0))
else:
    print("No feature points detected.")
```

```
Number of feature points detected: 32
```

```
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)

p1, status, error = cv2.calcOpticalFlowPyrLK(
    gray1,
    gray2,
    p0,
    None,
    **lk_params
)

if p1 is not None:
    good_new = p1[status == 1]
    good_old = p0[status == 1]
```

```
    print("Tracked points:", len(good_new))
else:
    print("Lucas-Kanade optical flow failed.")
```

```
Tracked points: 32
```

```
lk_output = frame2.copy()

for new, old in zip(good_new, good_old):

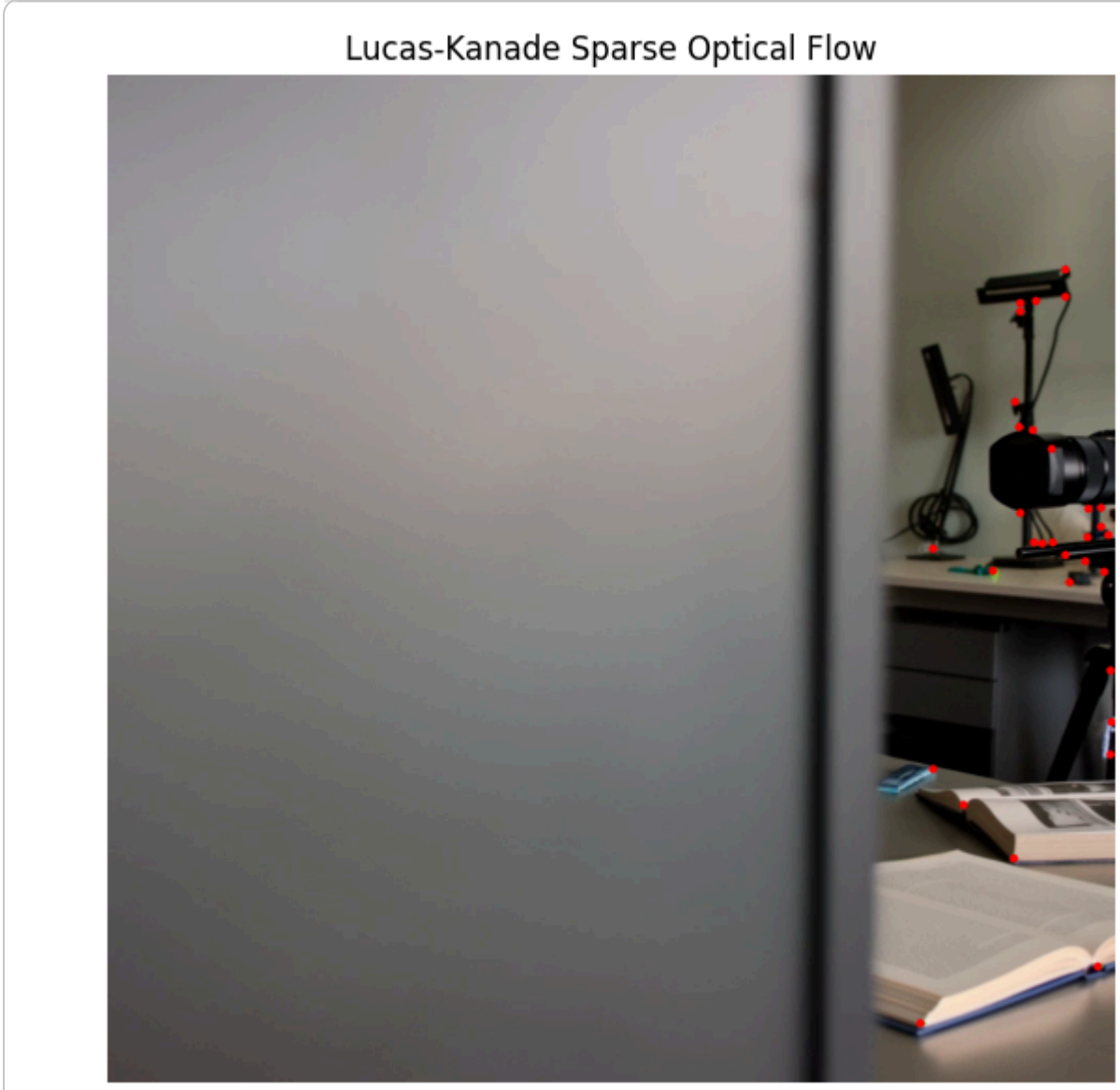
    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()

    cv2.arrowedLine(
        lk_output,
        (int(x_old), int(y_old)),
        (int(x_new), int(y_new)),
        (0, 255, 0),
        2,
        tipLength=0.3
    )

    cv2.circle(
        lk_output,
        (int(x_new), int(y_new)),
        4,
        (0, 0, 255),
        -1
    )

plt.figure(figsize=(12, 7))

plt.imshow(cv2.cvtColor(lk_output, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Sparse Optical Flow")
plt.axis("off")
plt.show()
```




```
flow = cv2.calcOpticalFlowFarneback(
    gray1,
    gray2,
    None,
    pyr_scale=0.5,
    levels=3,
    winsize=15,
    iterations=3,
    poly_n=5,
    poly_sigma=1.2,
    flags=0
)

print("Farneback Dense Optical Flow calculated successfully!")
print("Flow shape:", flow.shape)
```

```
Farneback Dense Optical Flow calculated successfully!
Flow shape: (960, 960, 2)
```

```
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

print("Minimum motion magnitude:", magnitude.min())
```

```
print("Maximum motion magnitude:", magnitude.max())
print("Average motion magnitude:", magnitude.mean())
```

```
Minimum motion magnitude: 4.057177e-18
Maximum motion magnitude: 7.884443
Average motion magnitude: 0.25924397
```

```
hsv = np.zeros_like(frame1)

hsv[..., 1] = 255

hsv[..., 0] = angle * 180 / np.pi / 2

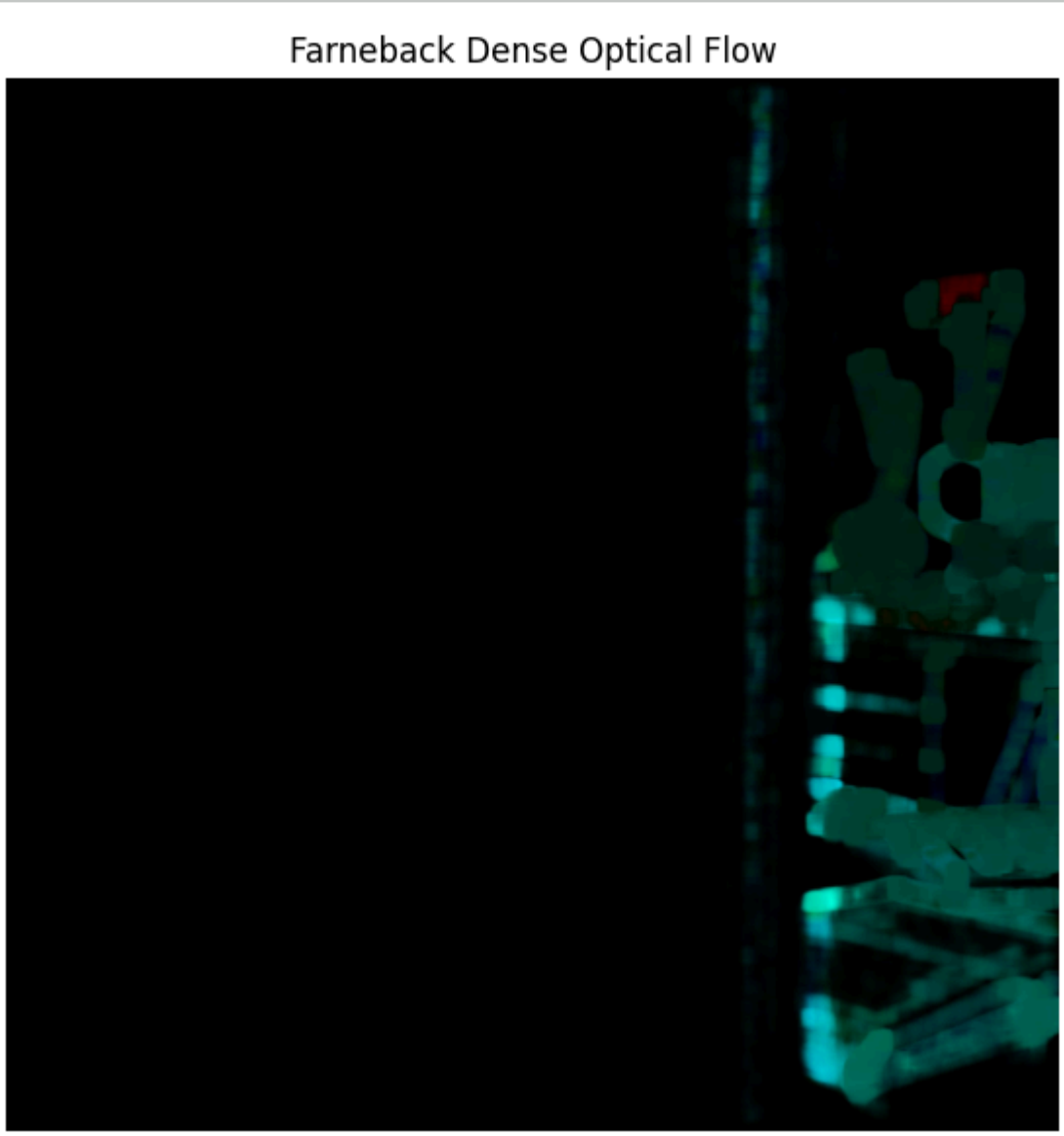
hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

dense_flow = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2RGB
)

plt.figure(figsize=(12, 7))

plt.imshow(dense_flow)
plt.title("Farneback Dense Optical Flow")
plt.axis("off")

plt.show()
```

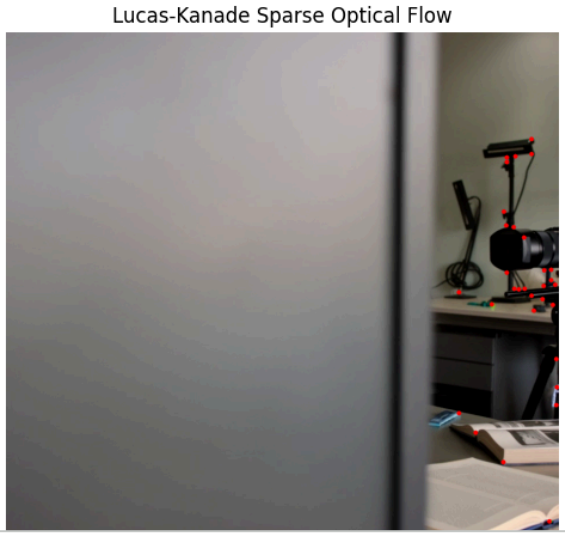


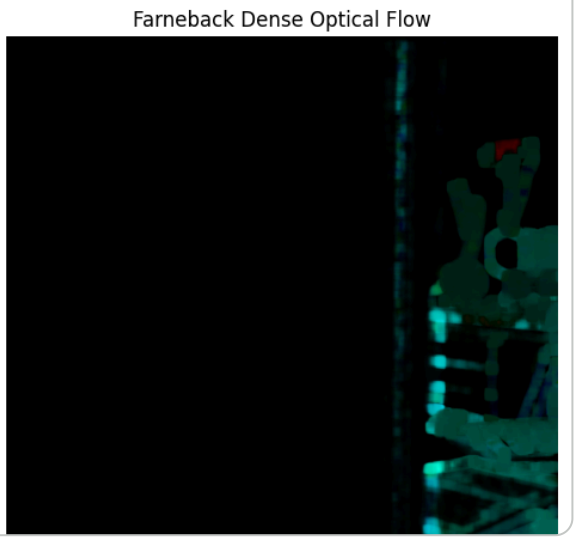

```
plt.figure(figsize=(15, 6))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(lk_output, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Sparse Optical Flow")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(dense_flow)
plt.title("Farneback Dense Optical Flow")
plt.axis("off")

plt.show()
```






```
avg_motion = np.mean(magnitude)
max_motion = np.max(magnitude)

print("Motion Analysis")
print("---------------------------")
print(f"Average Motion Magnitude: {avg_motion:.4f}")
print(f"Maximum Motion Magnitude: {max_motion:.4f}")

if avg_motion < 1:
    print("Observation: Very little motion is detected.")
elif avg_motion < 5:
    print("Observation: Moderate motion is detected.")
else:
    print("Observation: Significant motion is detected.")
```

```
Motion Analysis
---------------------------
Average Motion Magnitude: 0.2592
Maximum Motion Magnitude: 7.8844
Observation: Very little motion is detected.
```

```
step = 20

plt.figure(figsize=(12, 7))

plt.imshow(cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB))

h, w = gray1.shape

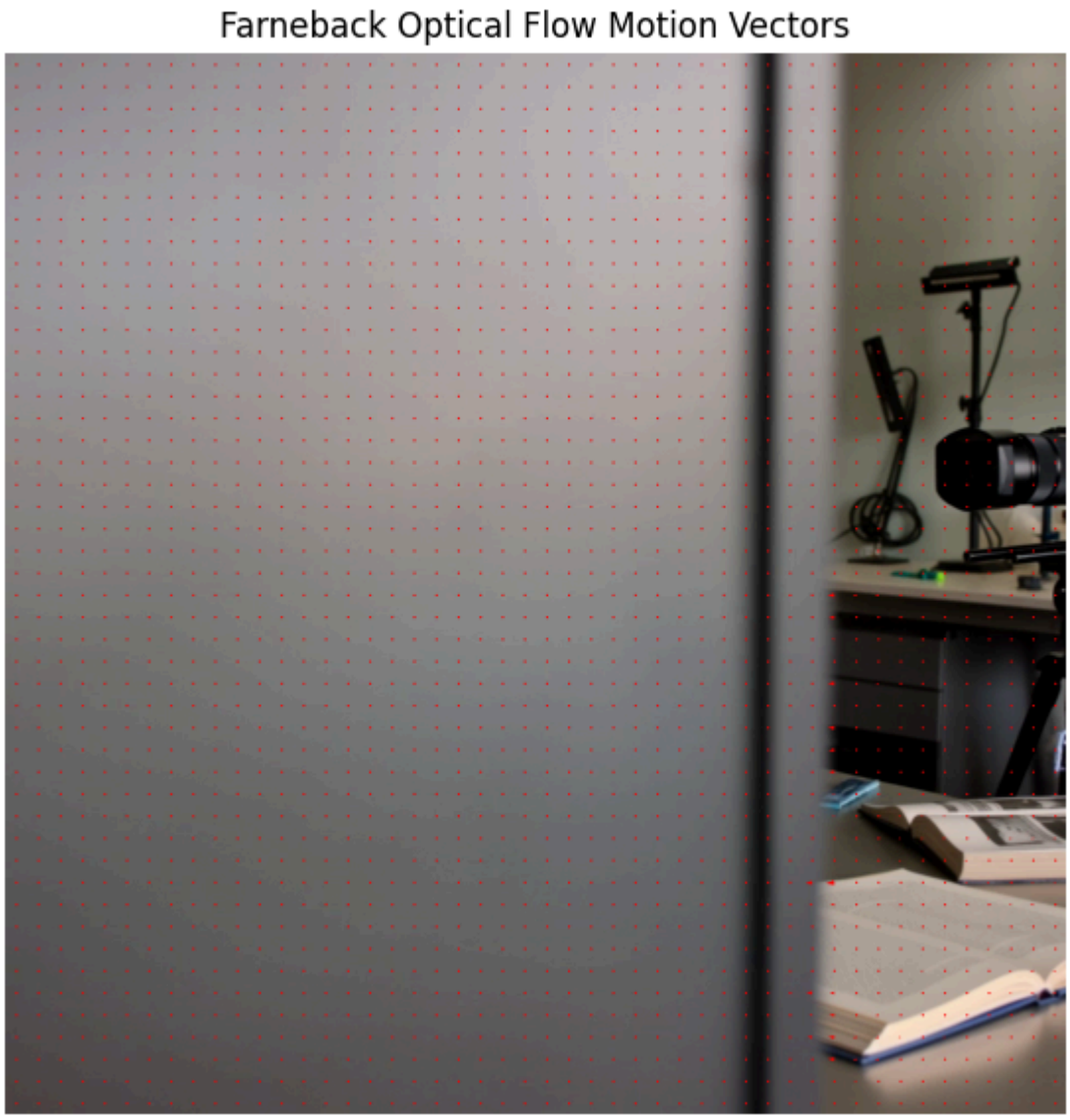
y, x = np.mgrid[
    step//2:h:step,
    step//2:w:step
].astype(int)

fx = flow[y, x, 0]
fy = flow[y, x, 1]

plt.quiver(
    x,
    y,
    fx,
    fy,
    color='red',
```

```
    angles='xy',
    scale_units='xy',
    scale=1
)

plt.title("Farneback Optical Flow Motion Vectors")
plt.axis("off")
plt.show()
```



Q1. What is Optical Flow? How is it used in computer vision?

Answer: Optical Flow is the apparent motion of objects, surfaces, or the camera between two consecutive frames of a video. It is represented using motion vectors that show the direction and amount of movement.

In computer vision, optical flow is used to detect and analyze motion. It is commonly used in object tracking, video surveillance, autonomous vehicles, robotics, gesture recognition, and action recognition.

Q2. Explain the working principle of the Lucas-Kanade Optical Flow algorithm.

Answer: Lucas-Kanade is a Sparse Optical Flow algorithm used to track selected feature points between consecutive frames.

It assumes that pixels in a small neighborhood have similar motion. First, important feature points are detected in the first frame. The algorithm then searches for their corresponding positions in the next frame using image intensity and gradients.

The displacement of these points is calculated and represented using motion vectors or arrows.

Main steps:

Detect important feature points. Take a small window around each point. Compare the window in consecutive frames. Calculate the movement of the feature. Display the movement using vectors. Q3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?

Answer: Dense Optical Flow estimates motion for almost every pixel in an image. It provides a detailed representation of the movement present in the scene.

Sparse Optical Flow Dense Optical Flow Tracks selected feature points Estimates motion across the image Uses fewer points Uses a large number of pixels Requires less computation Requires more computation Produces fewer motion vectors Produces a dense motion field Example: Lucas-Kanade Example: Farneback

Thus, Sparse Optical Flow is useful for feature tracking, while Dense Optical Flow is useful for detailed motion analysis.

Q4. Compare Lucas-Kanade and Farneback Optical Flow algorithms.

Answer:

Feature Lucas-Kanade Farneback Type Sparse Dense Motion estimation Selected feature points Almost every pixel Computational cost Lower Higher Speed Generally faster Generally slower Output Motion vectors at feature points Dense motion field Visualization Arrows/trajectories Color-coded motion Application Feature/object tracking Detailed motion analysis

Lucas-Kanade is useful when only important feature points need to be tracked. Farneback is useful when motion information over the complete image is required.

Q5. What assumptions are made while computing Optical Flow?

Answer: The main assumptions are:

Brightness constancy: The brightness of a moving point remains approximately constant between consecutive frames. Small motion: The movement between two consecutive frames is relatively small. Spatial smoothness: Neighboring pixels generally have similar motion. Object appearance remains similar: Objects do not change significantly between frames. Smooth motion: Motion generally changes gradually over nearby pixels.

These assumptions help the algorithm estimate the movement of objects between frames.

Q6. What factors can affect the accuracy of Optical Flow estimation?

Answer: Several factors can affect Optical Flow accuracy:

Object speed – Very fast objects can move significantly between two frames. Lighting changes – Changes in brightness can make tracking difficult. Camera movement – Movement of the camera can create motion throughout the image. Image texture – Textureless regions are difficult to track. Occlusion – Hidden objects cannot be tracked correctly. Motion blur – Fast

movement can blur important features. Frame rate – A low frame rate can produce large displacement. Noise and poor image quality – These can reduce the accuracy of motion estimation. Q7. Mention five real-world applications of Optical Flow.

Answer: Five applications of Optical Flow are:

Autonomous driving – Detecting and analyzing movement of vehicles and pedestrians. Video surveillance – Detecting and tracking moving objects. Object tracking – Following an object through consecutive video frames. Robotics – Helping robots understand movement in their environment. Gesture and action recognition – Detecting human movements and activities.